Design and Implementation of a Mini Unix Shell

Nabin Kumar Singh

March 2026

Abstract

This report presents the step-by-step design and implementation of a simplified Unix shell written in C. Instead of implementing the full shell at once, the system is constructed through a sequence of twelve programs, each introducing one fundamental concept of Unix systems programming. These include command execution, process creation, command parsing, pipes, and file descriptor redirection. The final program integrates all these components to create a functional mini shell capable of executing commands, supporting pipelines, and handling input and output redirection.

Contents

1 Introduction **2**

2 Background Concepts **2**

3 Project Design Philosophy **2**

4 Program Development Progression **3**
4.1 Basic Shell Loop (1gets.c) . . . 3
4.2 Executing Commands (2execlp.c) . . . 3
4.3 Command Parsing (3strtok.c and 4strtok.c) . . . 3
4.4 File Descriptor Redirection (5dup.c) . . . 4
4.5 Basic Pipes (6pipe.c) . . . 4
4.6 Two-Process Pipelines (7pipe.c) . . . 4
4.7 User Pipe Commands (8pipe.c) . . . 4
4.8 Flexible Command Execution (9pipe.c and 10pipe.c) . . . 4
4.9 Output Redirection (11pipe.c) . . . 4
4.10 Final Shell Implementation (12shell.c) . . . 5

5 Final Shell Architecture **5**
5.1 Example Shell Execution . . . 5

6 Limitations **6**

7 Future Improvements **6**

8 Conclusion **7**

1 Introduction

A Unix shell is a command-line interpreter that provides an interface between the user and the operating system. It allows users to execute programs, manage files, and combine commands using mechanisms such as pipes and input/output redirection. Popular shells such as `bash`, `zsh`, and `sh` provide a rich set of features that enable powerful command-line workflows.

Understanding how a shell operates internally provides valuable insight into fundamental operating system concepts such as process creation, program execution, file descriptor manipulation, and inter-process communication. Rather than implementing a complete shell in a single step, this project approaches the problem incrementally.

The shell is developed through a sequence of twelve programs, each introducing a specific concept of Unix systems programming. Starting from a basic program that reads user input, the implementation gradually evolves to support command execution, argument parsing, pipes, and input/output redirection. Each stage builds upon the previous one, allowing a clear understanding of how the components of a shell interact.

The final program integrates all previously developed concepts into a simplified Unix-like shell capable of executing commands, handling pipelines, and performing file redirection. This incremental approach highlights both the architectural structure of shells and the underlying system calls that make them possible.

2 Background Concepts

Several core Unix system concepts are required to implement a command-line shell.

Process Creation. Unix systems create new processes using the `fork()` system call. This call duplicates the current process, creating a parent and a child process. The child process can then execute a different program.

Program Execution. The `exec()` family of system calls replaces the current process image with a new program. Functions such as `execlp()` and `execvp()` are commonly used to execute commands entered by the user.

Inter-Process Communication. Pipes allow the output of one process to become the input of another. The `pipe()` system call creates a pair of file descriptors that enable data transfer between processes.

File Descriptor Redirection. Unix systems represent input and output streams as file descriptors. Using the `dup2()` system call, a process can redirect standard input or output to files or pipes.

Command Parsing. User commands must be separated into tokens before execution. Functions such as `strtok()` can be used to split the command string into arguments that are passed to `exec()` functions.

3 Project Design Philosophy

Implementing a Unix shell involves several independent system programming concepts, including process creation, command parsing, file descriptor manipulation, and inter-process communication. Attempting to implement all of these mechanisms in a single program would make the development process difficult to understand and debug.

To address this challenge, the shell in this project is implemented using an incremental development approach. Instead of constructing the complete shell at once, the implementation is divided into a sequence of smaller programs, each introducing one fundamental concept of Unix systems programming. Every program builds directly on the ideas introduced in the previous stage, gradually expanding the functionality of the shell.

The development begins with a simple program that reads user input and echoes it back to the terminal. Once the basic shell loop is established, the next step introduces process creation and command execution using the `fork()` and `exec()` system calls. After command execution is working, additional features are implemented step by step, including command parsing, argument support, pipes for inter-process communication, and input/output redirection.

This incremental approach offers several advantages. First, it isolates individual concepts so that each feature can be understood independently. Second, it simplifies debugging because each stage can be tested

before introducing additional complexity. Finally, it mirrors how many real systems are designed in practice, where complex software is constructed by gradually integrating smaller components.

By the time the final program is reached, all of the previously developed concepts are combined to form a simplified Unix shell capable of executing commands, supporting pipelines, and performing file redirection.

4 Program Development Progression

The shell developed in this project was constructed incrementally through a series of twelve programs. Each program introduces a new concept of Unix systems programming and extends the functionality of the previous version. Rather than implementing all shell features in a single step, this staged development approach allows each component of the shell to be understood independently.

The progression begins with a simple interactive loop that reads commands from the user. Subsequent programs introduce command execution, argument parsing, pipe-based inter-process communication, and input/output redirection. At each stage, the shell becomes progressively more capable while maintaining the structure established in earlier programs.

By the final stage of the development process, all previously introduced features are integrated into a simplified Unix shell capable of executing commands, handling pipelines, and performing file redirection.

The following subsections describe the functionality introduced at each stage of the implementation.

4.1 Basic Shell Loop (1gets.c)

The first program establishes the basic interactive behavior of a shell. It repeatedly prompts the user for input, reads a command from the terminal, and prints the entered command back to the screen. This is implemented using a continuous loop that waits for user input before proceeding to the next iteration.

Although the program does not execute commands yet, it demonstrates the core structure used by most shells: an infinite loop that reads input, processes the command, and returns control to the user. This program therefore provides the foundation upon which all later stages of the shell implementation are built.

4.2 Executing Commands (2execlp.c)

The second program extends the basic shell loop by allowing the execution of external commands. This is achieved using the Unix system calls `fork()`, `execlp()`, and `wait()`.

The `fork()` system call creates a new child process that is an exact copy of the parent process. After the fork, the child process replaces its program image using `execlp()`, which loads and executes the requested command. Meanwhile, the parent process waits for the child to complete using `wait()` before returning to the shell prompt.

This mechanism reflects the standard way Unix shells execute commands: the shell creates a child process for each command and uses the `exec` family of functions to run the requested program. With this step, the shell is now capable of executing simple commands entered by the user.

4.3 Command Parsing (3strtok.c and 4strtok.c)

The next stage introduces command parsing so that the shell can handle commands with arguments. When a user enters a command such as `ls -l` or `grep txt file.txt`, the input string must be separated into individual tokens before execution.

This program uses the `strtok()` function to split the input command string into separate words using spaces as delimiters. The resulting tokens are stored in an array of character pointers, which represents the argument list for the command.

Once the command and its arguments are properly separated, the program uses `execvp()` to execute the command. Unlike `execlp()`, which requires arguments to be passed individually, `execvp()` accepts an array of arguments, making it well suited for commands with multiple parameters.

With this stage completed, the shell is now capable of executing commands that include arguments, bringing it closer to the behavior of a real Unix shell.

4.4 File Descriptor Redirection (5dup.c)

This stage introduces the concept of file descriptor redirection, which is commonly used in Unix shells to control where input and output are sent. In Unix systems, standard input, output, and error are represented by file descriptors 0, 1, and 2 respectively.

The program demonstrates how the `dup2()` system call can be used to duplicate one file descriptor onto another. By opening a file and redirecting the standard output descriptor to this file using `dup2()`, the program causes any output that would normally appear on the terminal to be written into the file instead.

This mechanism forms the basis of output redirection used in shells, such as commands like `ls > output.txt`. Understanding file descriptor manipulation is therefore essential for implementing more advanced shell features in later stages of the project.

4.5 Basic Pipes (6pipe.c)

This stage introduces inter-process communication using Unix pipes. The `pipe()` system call creates a unidirectional communication channel between two processes and returns two file descriptors: one for reading and one for writing.

Using `dup2()`, the program redirects the standard output of one process to the write end of the pipe, while another process reads from the read end of the pipe through its standard input. In this way, the output produced by one program can be used directly as the input to another program.

This mechanism forms the basis of pipeline commands commonly used in Unix shells, such as `ls | sort`.

4.6 Two-Process Pipelines (7pipe.c)

After introducing pipes, the next stage implements a simple two-process pipeline. Two child processes are created using `fork()`, where the first process produces output and the second process consumes it through the pipe.

To ensure correct communication, unused ends of the pipe are closed in each process. This prevents unintended blocking and guarantees that data flows correctly between the commands.

With this structure in place, the shell is now capable of executing commands connected by a pipeline.

4.7 User Pipe Commands (8pipe.c)

The next improvement allows the shell to interpret pipeline commands entered by the user. The program detects the pipe symbol (`|`) in the command line and separates the input into two independent commands.

Each command is then tokenized into arguments and executed in separate child processes, while the pipe connects the output of the first command to the input of the second command. This allows the shell to support user commands such as `ls | wc`.

4.8 Flexible Command Execution (9pipe.c and 10pipe.c)

At this stage the shell becomes more flexible by supporting both piped and non-piped commands. The program analyzes the user input to determine whether a pipe symbol is present in the command line.

If a pipe is detected, the shell constructs a pipeline using two child processes connected by a pipe. If no pipe symbol is found, the command is executed normally using the standard `fork()` and `exec()` mechanism.

This allows the shell to support both simple commands such as `ls -l` and piped commands such as `ls | sort` within the same execution framework.

4.9 Output Redirection (11pipe.c)

The next improvement introduces output redirection. The program detects the redirection symbol `>` in the command line and extracts the file name that follows it.

The file is opened using the `open()` system call, and the standard output file descriptor is redirected to this file using `dup2()`. As a result, any output produced by the executed command is written to the specified file instead of being displayed on the terminal.

This enables commands such as `ls > files.txt`, which are commonly used in Unix shells to store command output.

4.10 Final Shell Implementation (12shell.c)

The final program integrates all the previously developed concepts into a single working shell. The shell reads user commands, parses the input, detects pipes and redirection symbols, and executes the corresponding programs using child processes.

The implementation supports command execution with arguments, pipelines between two commands, and basic input/output redirection. By combining process creation, command parsing, pipes, and file descriptor manipulation, the program demonstrates how a simplified Unix shell can be constructed using fundamental operating system mechanisms.

Although the shell does not implement advanced features such as command history or job control, it captures the essential behavior of a Unix command interpreter and illustrates how these capabilities are built from core system calls.

5 Final Shell Architecture

The final shell follows a structured sequence of steps to process and execute user commands. When a command is entered, the shell first reads the input from the terminal and performs tokenization to separate the command and its arguments.

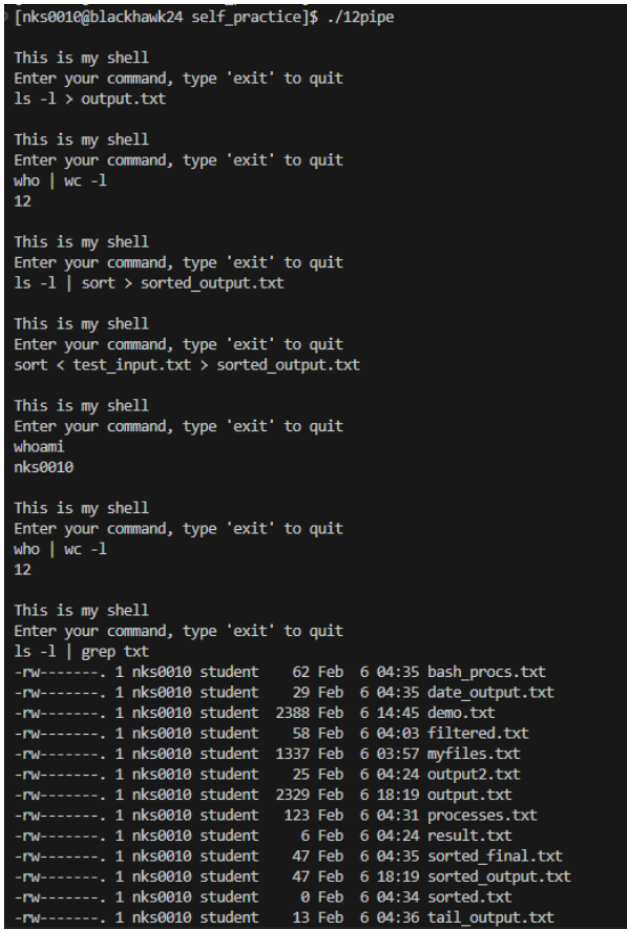
The shell then checks whether the command contains special symbols such as pipes (`|`) or redirection operators (`<` and `>`). Depending on the detected symbols, the shell prepares the necessary pipes or file descriptor redirections.

After parsing the command, the shell creates one or more child processes using the `fork()` system call. Each child process replaces its program image using the `execvp()` function to run the requested command. If pipes or redirection are involved, the shell uses `dup2()` to connect the appropriate file descriptors before executing the program.

Finally, the parent process waits for the child processes to complete before returning control to the shell prompt, allowing the user to enter the next command.

5.1 Example Shell Execution

To verify the functionality of the implemented shell, several commands were executed demonstrating features such as command execution, pipelines, and input/output redirection. The shell successfully supports common Unix-style operations including command chaining using pipes and redirecting output to files.



```
[nks0010@blackhawk24 self_practice]$ ./12pipe

This is my shell
Enter your command, type 'exit' to quit
ls -l > output.txt

This is my shell
Enter your command, type 'exit' to quit
who | wc -l
12

This is my shell
Enter your command, type 'exit' to quit
ls -l | sort > sorted_output.txt

This is my shell
Enter your command, type 'exit' to quit
sort < test_input.txt > sorted_output.txt

This is my shell
Enter your command, type 'exit' to quit
whoami
nks0010

This is my shell
Enter your command, type 'exit' to quit
who | wc -l
12

This is my shell
Enter your command, type 'exit' to quit
ls -l | grep txt
-rw-------. 1 nks0010 student    62 Feb  6 04:35 bash_procs.txt
-rw-------. 1 nks0010 student    29 Feb  6 04:35 date_output.txt
-rw-------. 1 nks0010 student  2388 Feb  6 14:45 demo.txt
-rw-------. 1 nks0010 student    58 Feb  6 04:03 filtered.txt
-rw-------. 1 nks0010 student  1337 Feb  6 03:57 myfiles.txt
-rw-------. 1 nks0010 student    25 Feb  6 04:24 output2.txt
-rw-------. 1 nks0010 student  2329 Feb  6 18:19 output.txt
-rw-------. 1 nks0010 student   123 Feb  6 04:31 processes.txt
-rw-------. 1 nks0010 student     6 Feb  6 04:24 result.txt
-rw-------. 1 nks0010 student    47 Feb  6 04:35 sorted_final.txt
-rw-------. 1 nks0010 student    47 Feb  6 18:19 sorted_output.txt
-rw-------. 1 nks0010 student     0 Feb  6 04:34 sorted.txt
-rw-------. 1 nks0010 student    13 Feb  6 04:36 tail_output.txt
```

Figure 1: Example execution of the implemented shell demonstrating pipes and redirection.

The example shown above includes commands such as `who | wc -l`, `ls -l > output.txt`, and `ls -l | sort > sorted_output.txt`. These examples confirm that the shell correctly handles process creation, pipe communication, and file descriptor redirection.

6 Limitations

Although the implemented shell demonstrates the core mechanisms used in Unix command interpreters, it remains a simplified implementation. Several features commonly found in modern shells are not included.

First, the shell currently supports pipelines between only two commands. More complex pipelines involving multiple commands are not implemented. Second, the shell does not support background execution using the `&` operator. All commands run in the foreground and the shell waits for their completion.

Additionally, advanced features such as command history, tab completion, environment variable expansion, and built-in commands like `cd` are not implemented. These features would require additional parsing logic and process management mechanisms.

7 Future Improvements

Although the implemented shell successfully demonstrates the core mechanisms of Unix command execution, several improvements could be introduced to make the system more complete and closer to real Unix shells.

One possible enhancement is support for pipelines involving more than two commands. Currently, the shell handles only a single pipe, while real shells allow chains such as `cmd1 | cmd2 | cmd3 | cmd4`.

Another improvement would be support for background execution using the `&` operator. This would allow the shell to start a process without waiting for its completion, enabling concurrent execution of multiple programs.

Additional features such as command history, tab completion, and built-in commands like `cd` or `exit` could also be added. These capabilities would require more advanced command parsing and process management.

Implementing these features would extend the shell beyond a basic educational implementation and bring it closer to the functionality of modern Unix command interpreters.

8 Conclusion

This project demonstrates the incremental development of a simplified Unix shell using fundamental systems programming concepts. Rather than building the entire shell at once, the implementation was constructed through a sequence of twelve programs, each introducing an important component such as command execution, argument parsing, pipes, and file descriptor redirection.

By gradually extending the functionality from a basic input loop to a fully functional shell, the project illustrates how core Unix system calls such as `fork()`, `execvp()`, `pipe()`, and `dup2()` work together to support command-line execution.

Although the final implementation does not include advanced features found in modern shells, it captures the essential mechanisms that enable command execution and inter-process communication in Unix systems. The step-by-step design also provides a clear understanding of how complex systems can be built by progressively integrating smaller components.